

软件过程改进方案设计文档

提交人：唐立峰

学号：2023141461045

课程：软件过程与管理

项目名称：面向网课的智能监督系统

一、改进背景

本项目为“面向网课的智能监督系统”，主要面向线上课程学习场景，目标是通过摄像头视频流对学生上网课过程中的状态进行分析，并将分析结果进行展示。系统整体包括视频采集与预处理、特征提取、状态估计、界面展示等模块。其中，我主要负责视频采集与预处理模块，该模块需要完成视频流读取、图像帧处理、人脸区域信息整理，以及向后续特征提取模块提供统一格式的数据。

在第一次软件过程定义作业中，我已经结合本项目特点，将软件过程划分为项目计划、需求分析、概要设计、详细设计与编码实现、集成与测试、文档与配置管理、评审与提交等阶段。这个过程适合课程小组项目，能够覆盖项目从计划到提交的主要活动。但是，随着生成式AI、大语言模型、AI代码助手和智能测试工具的发展，原有软件过程仍然可以进一步改进。

本次改进的核心思路不是推翻原有过程，而是在原有软件生命周期各阶段中加入AI辅助节点，形成“人工主导、AI辅助、人工审核、结果验证”的智能化软件过程。AI技术主要用于提高需求整理、接口设计、代码生成、测试设计和文档管理的效率；人工成员仍然负责最终判断、修改、验证和提交。

二、改进目标

本次软件过程改进主要有以下目标：

1. 提高需求分析效率和完整性

利用AI辅助整理项目需求，补充功能需求、非功能需求、异常场景和验收标准，减少人工遗漏。

2. 提高概要设计和接口设计质量

利用AI辅助生成模块说明、接口字段表和流程图，重点保证视频采集与预处理模块和后续模块之间的数据格式一致。

3. 提高编码实现效率

利用AI代码助手生成基础代码框架、异常处理逻辑和代码注释，减少重复性编码工作。

4. 提高测试覆盖度

利用AI生成测试用例，补充正常场景、异常场景、边界场景和性能场景，避免只测试“能不能跑通”。

5. 提高文档一致性和可维护性

利用AI检查不同文档中的模块名称、字段名称和过程描述是否一致，减少最终提交前的文档返工。

6. 建立AI输出审核机制

明确AI不能替代人工决策，所有AI生成的需求、设计、代码、测试用例和文档内容都必须经过人工审核。

三、改进后的总体软件过程

改进后的软件过程仍然以原有过程为主线，但在关键环节加入AI辅助活动。总体流程如下：

项目计划阶段

需求分析阶段

AI辅助需求整理与遗漏检查

人工审核并确认需求

概要设计阶段

AI辅助接口设计与流程图生成

人工评审模块边界和数据流

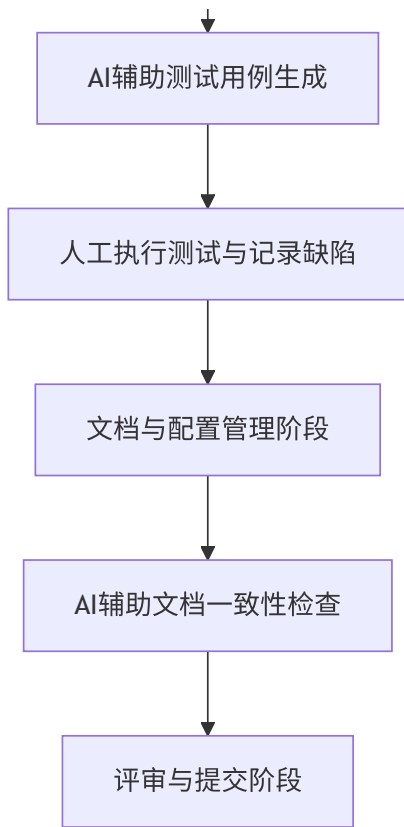
详细设计与编码实现阶段

AI辅助代码生成与代码审查

人工修改、运行和确认

集成与测试阶段





该流程体现了两个原则：

第一，AI嵌入原有过程，而不是取代原有过程。原有的软件过程阶段仍然保留，只是在关键活动中增加AI辅助节点。

第二，AI输出必须经过人工审核。AI生成的内容只能作为初稿或建议，不能直接作为最终提交物。

四、AI辅助需求分析过程设计

4.1 原有过程存在的问题

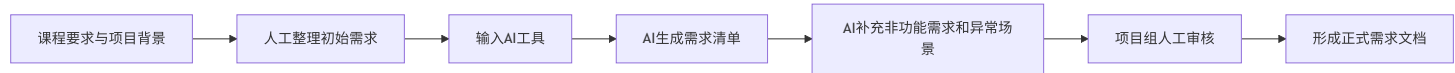
原有需求分析主要依靠组内讨论和人工整理。对于课程项目来说，这种方式比较直接，但容易出现以下问题：

- 需求描述不够结构化；
- 功能需求和非功能需求区分不清；
- 异常场景考虑不足；
- 模块之间的输入输出边界不够清楚；
- 需求和后续测试之间缺少对应关系。

例如，本项目可能会明确“系统需要通过摄像头分析学生状态”，但如果不进一步细化，就可能遗漏摄像头不可用、视频帧为空、无人脸、多人人脸、帧率过低等情况。

4.2 改进后的活动流程

改进后的需求分析过程如下：



4.3 AI参与方式

在需求分析阶段，AI主要参与以下活动：

- 1. 根据项目背景生成需求清单；
- 2. 将需求划分为功能需求、非功能需求、数据需求和异常需求；
- 3. 根据需求生成验收标准；
- 4. 检查需求是否存在表述模糊或冲突；
- 5. 补充人工容易忽略的异常场景。

4.4 AI输出物质量标准

检查项	质量标准
完整性	覆盖视频采集、预处理、特征提取、状态估计、界面展示等核心功能
一致性	模块名称、字段名称与项目已有文档一致
可实现性	不提出明显超出课程项目能力范围的功能
可验证性	每条需求应能够对应测试用例或验收标准
准确性	不编造项目不存在的模块或技术路线

4.5 推荐工具

工具名称	核心功能	与现有过程的集成方式	预期效率提升
ChatGPT	需求整理、需求补充、验收标准生成	输入项目背景和初始需求，生成需求文档初稿	需求初稿编写时间减少约40%
Kimi / 通义千问	长文档总结、需求归纳	输入课程要求和已有文档，辅助总结关键需求	文档整理效率提升约30%
Markdown	结构化记录需求	将AI生成内容整理为Markdown文档	提高文档规范性

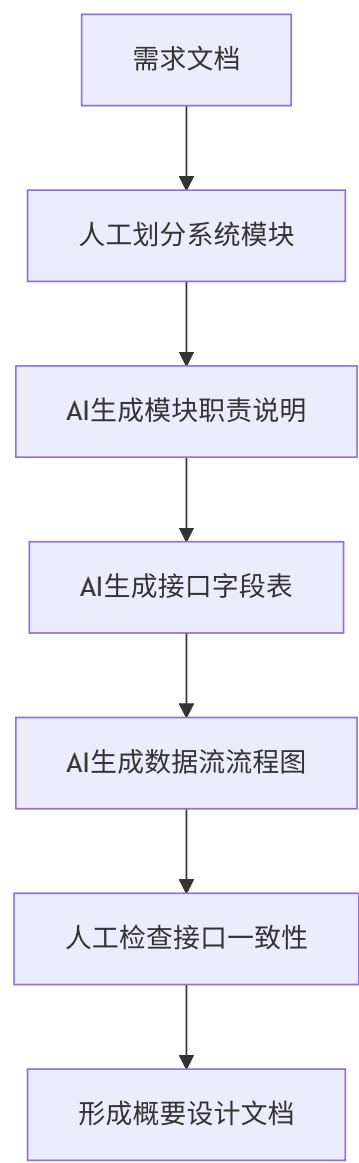
五、AI辅助概要设计过程设计

5.1 原有过程存在的问题

概要设计阶段需要明确系统模块、模块职责、输入输出、接口字段和异常处理方式。对于本项目而言，视频采集与预处理模块和后续特征提取模块之间的数据接口非常关键。如果接口字段不清楚，后续集成时容易出现字段缺失、类型不一致或含义理解不同的问题。

例如，预处理模块需要输出 `timestamp`、`frame`、`faces`、`owner_face_id` 等字段。如果这些字段没有明确类型和异常处理方式，后续模块就难以稳定调用。

5.2 改进后的活动流程



5.3 AI参与方式

在概要设计阶段，AI主要参与以下活动：

1. 根据需求文档生成模块职责说明；
2. 辅助生成模块之间的数据流图；
3. 生成接口字段表；
4. 检查字段类型、含义和异常情况是否完整；
5. 根据文字描述生成Mermaid流程图。

5.4 接口设计示例

以视频采集与预处理模块为例，改进后的接口设计可以如下：

字段名	类型	含义	是否必填	异常处理
timestamp	float	当前帧时间戳	是	若无法获取，则记录当前系统时间或异常状态
frame	ndarray	当前图像帧	是	空帧时返回错误状态，不继续处理
faces	list	人脸检测结果列表	是	无人脸时为空列表
face_id	int / str	人脸编号	否	无法识别时可为空
face_roi	tuple / list	人脸区域坐标	否	坐标越界时进行修正或丢弃
owner_face_id	int / str / None	主体人脸编号	否	无法确定主体时为None

5.5 AI输出物质量标准

检查项	质量标准
接口完整性	字段名、类型、含义、异常处理齐全
逻辑正确性	模块之间的数据流符合实际处理顺序
可读性	表格和流程图清晰，便于其他成员理解
可实现性	不引入课程项目无法实现的复杂架构
可追踪性	每个接口字段都能对应到需求或模块功能

5.6 推荐工具

工具名称	核心功能	与现有过程的集成方式	预期效率提升
ChatGPT	生成模块说明、接口表和流程图	输入需求说明和模块划分，生成概要设计初稿	设计文档编写时间减少约30%
Mermaid	绘制流程图	在Markdown中嵌入流程图代码	提高设计文档可视化程度
draw.io / Visio	绘制正式图表	根据AI生成的流程进行人工美化	提高展示效果

六、AI辅助编码实现过程设计

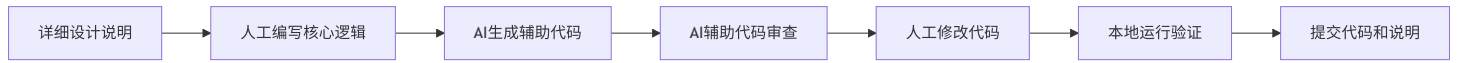
6.1 原有过程存在的问题

在原有编码过程中，各成员根据分工独立完成模块代码。该方式比较灵活，但也存在以下问题：

- 不同成员代码风格不统一；
- 接口字段命名可能不一致；
- 异常处理不完整；

- 重复性代码较多；
- 调试错误时耗时较长；
- 缺少代码审查环节。

6.2 改进后的活动流程



6.3 AI参与方式

在编码实现阶段，AI主要参与以下活动：

1. 根据函数说明生成代码框架；
2. 根据接口字段生成数据封装逻辑；
3. 生成异常处理代码；
4. 解释运行报错；
5. 检查代码命名和注释；
6. 提出重构建议；
7. 生成简单单元测试代码。

对于视频采集与预处理模块，AI可以辅助生成摄像头读取、空帧判断、人脸区域数据封装、日志记录等基础代码。但最终代码必须由开发人员自己理解、修改和运行。

6.4 AI输出物质量标准

检查项	质量标准
可运行性	AI生成代码必须经过本地运行验证
接口一致性	输入输出字段必须与概要设计文档一致
可读性	函数命名清楚，必要位置有注释
安全性	不包含无关的外部数据上传或危险操作
可维护性	代码结构清晰，避免过长函数和重复逻辑

6.5 推荐工具

工具名称	核心功能	与现有过程的集成方式	预期效率提升
GitHub Copilot	代码补全、代码生成、测试生成、重构建议	集成到VS Code或JetBrains IDE中辅助编码	基础代码编写效率提升约30%
ChatGPT	代码解释、错误分析、函数生成	将局部代码和报错信息输入AI进行分析	调试时间减少约30%
通义灵码 / CodeGeeX	中文环境下代码补全与解释	作为IDE插件辅助学生开发	提高编码便利性

七、AI辅助测试验证过程设计

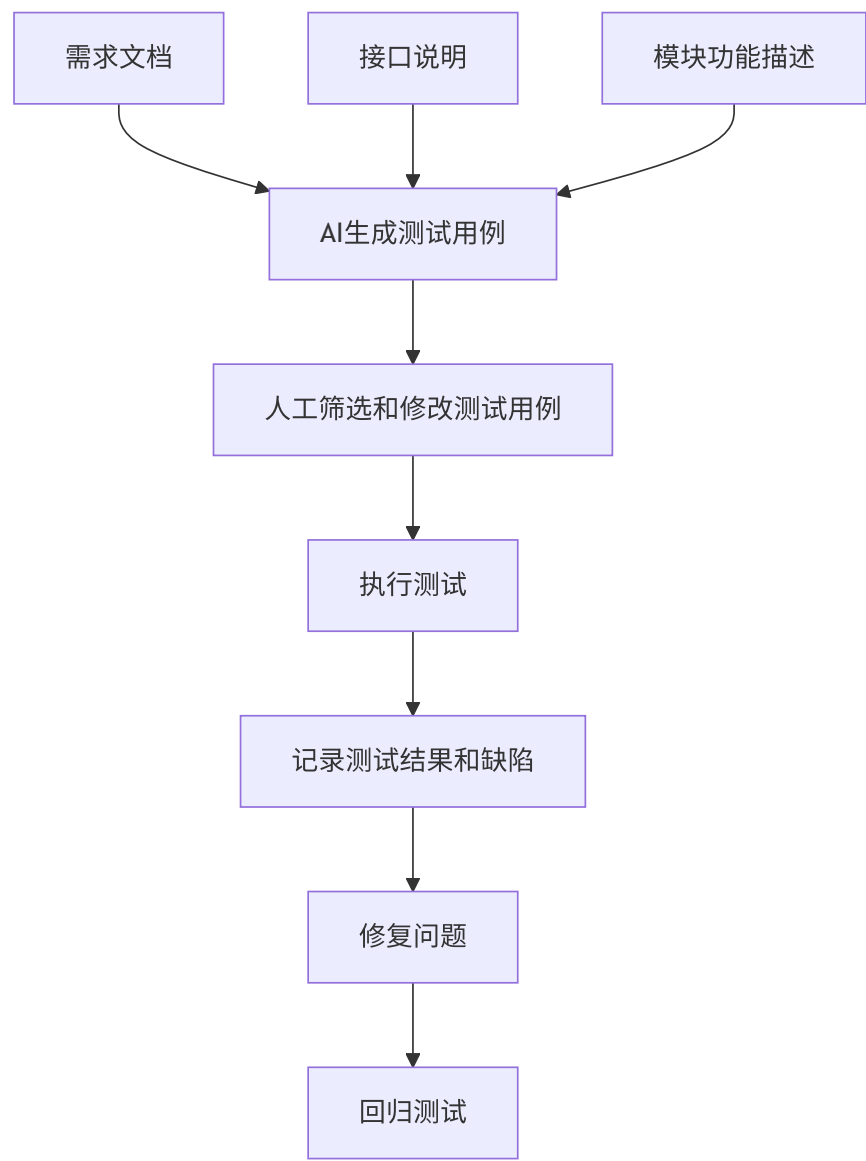
7.1 原有过程存在的问题

测试验证阶段是课程项目中容易被压缩的环节。原有测试通常偏重正常流程，例如检查摄像头是否能打开、系统是否能运行、界面是否能显示结果。但对于实际系统而言，异常场景和边界场景同样重要。

对于本项目的视频采集与预处理模块，需要考虑：

- 摄像头无法打开；
- 视频帧为空；
- 当前画面无人脸；
- 当前画面多人脸；
- 人脸框超出图像边界；
- 输出字段缺失；
- 帧率过低；
- 连续多帧处理不稳定。

7.2 改进后的活动流程



7.3 AI参与方式

在测试阶段，AI主要参与以下活动：

- 1. 根据需求和接口说明生成测试用例；
- 2. 补充异常场景和边界场景；
- 3. 为测试用例设置优先级；
- 4. 生成测试数据建议；
- 5. 根据测试结果辅助分析缺陷原因。

7.4 测试用例分类

测试类别	示例
正常场景	摄像头正常打开，画面中存在单人脸

测试类别	示例
边界场景	多人脸、人脸位于画面边缘
异常场景	摄像头不可用、视频帧为空、无人脸
数据格式场景	faces 为空、face_roi 格式错误、字段缺失
性能场景	连续处理多帧时延迟过高或帧率过低

7.5 AI输出物质量标准

检查项	质量标准
覆盖度	覆盖正常、异常、边界、性能场景
可执行性	每条用例有明确输入、步骤和预期结果
相关性	与当前模块功能直接相关
可复现性	测试步骤清楚，可重复执行
缺陷发现能力	能帮助发现接口和异常处理问题

7.6 推荐工具

工具名称	核心功能	与现有过程的集成方式	预期效率提升
ChatGPT	根据接口说明生成测试用例	输入模块功能和字段定义，生成测试用例表	测试用例设计时间减少约40%
PyTest	Python单元测试框架	根据AI生成用例编写自动化测试脚本	提高测试执行效率
GitHub Copilot	生成单元测试代码	根据已有函数生成测试函数	测试代码编写效率提升约30%
Markdown表格	记录测试用例和测试结果	统一记录测试编号、输入、预期和结果	提高测试文档规范性

八、AI辅助文档与配置管理过程设计

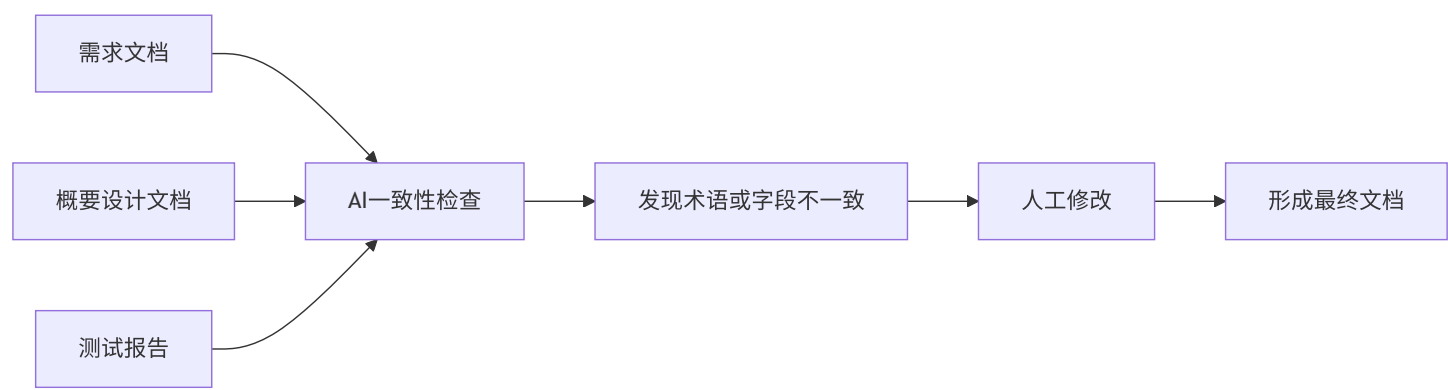
8.1 原有过程存在的问题

项目推进过程中需要维护项目计划、需求分析文档、概要设计文档、周报、测试报告和最终提交材料。由于文档较多，且由不同成员参与编写，容易出现以下问题：

- 模块名称前后不一致；
- 接口字段命名不一致；
- 文档内容与实际代码不一致；
- 周报和最终总结缺少衔接；

- 格式不统一。

8.2 改进后的活动流程



8.3 AI参与方式

AI可以辅助完成：

1. 检查不同文档中的字段名称是否一致；
2. 检查需求、设计和测试之间是否对应；
3. 润色文档表达；
4. 根据工作记录生成周报初稿；
5. 辅助形成最终总结。

8.4 AI输出物质量标准

检查项	质量标准
真实性	不编造项目没有完成的内容
一致性	模块名称、字段名称前后一致
格式规范性	符合Markdown书写格式
可读性	表述清楚，层次分明
可追踪性	需求、设计、测试之间能够对应

8.5 推荐工具

工具名称	核心功能	与现有过程的集成方式	预期效率提升
ChatGPT	文档润色、一致性检查、总结生成	输入文档片段，检查前后逻辑和字段	文档整理时间减少约30%
GitHub	文档和代码版本管理	通过提交记录管理不同版本	提高配置管理规范性
Markdown	统一文档格式	所有文档均采用Markdown编写并导出PDF	提高格式统一性

九、新增AI相关角色与职责

由于本项目是课程小组项目，团队规模较小，因此不需要设置复杂的企业级岗位，而是将AI相关职责分配给现有成员兼任。

角色	主要职责	对应阶段
项目负责人	制定计划、协调进度、组织评审和最终提交	全过程
需求分析人员	整理需求，审核AI生成的需求内容	需求分析阶段
设计负责人	确认模块划分、接口字段和流程图	概要设计阶段
开发人员	编写代码，审核AI生成代码并运行验证	编码实现阶段
测试人员	筛选AI生成测试用例，执行测试并记录缺陷	测试验证阶段
AI协同负责人	选择AI工具，记录AI使用过程，检查AI输出质量	全过程
文档负责人	维护Markdown文档，检查格式和内容一致性	文档管理阶段

其中，AI协同负责人是本次改进中新增的关键角色。该角色需要保证AI工具使用过程可记录、可复现、可验证，避免出现“直接复制AI输出但无人负责”的情况。

十、过程度量指标设计

为了判断AI引入后是否真正提升了过程质量，需要设置相应度量指标。

指标类别	指标名称	含义	示例计算方式
效率指标	需求文档初稿生成时间	需求初稿从无到有所需时间	人工耗时与AI辅助耗时对比
效率指标	测试用例设计时间	生成测试用例所需时间	分钟数对比
质量指标	测试场景覆盖数	覆盖正常、异常、边界、性能场景数量	场景类别数量统计
质量指标	接口字段一致率	文档和代码中字段名称是否一致	一致字段数 / 总字段数
质量指标	缺陷发现数量	测试阶段发现的问题数量	缺陷记录条数
AI贡献度指标	AI建议采纳率	AI生成内容中被人工采纳的比例	采纳条数 / AI生成条数
AI贡献度指标	AI输出修改率	AI输出内容需要人工修改的比例	修改内容 / AI输出内容
风险指标	AI错误输出数量	AI生成的错误、虚构或不可用内容数量	人工审核发现的问题数

这些指标可以用于后续验证报告中，对AI辅助方式和传统人工方式进行对比。

十一、风险评估与应对策略

11.1 数据安全和隐私风险

本项目涉及摄像头视频流和人脸区域信息。如果将真实视频帧、人脸图像或个人隐私数据上传到在线AI工具，可能存在隐私泄露风险。

应对策略：

- 不向在线AI工具上传真实人脸图像或视频；
- 只输入抽象后的模块说明、接口字段和错误信息；
- 测试时优先使用模拟数据或公开样例；
- 文档中避免包含真实个人隐私信息。

11.2 AI幻觉风险

AI可能生成看似合理但实际不存在的功能、工具、接口或数据。例如，AI可能建议引入复杂的云端监控平台或自动扩容机制，但这些内容并不适合课程项目。

应对策略：

- 所有AI输出必须经过人工审核；
- 超出课程范围的建议不采纳；
- 需求、设计、代码和测试均以实际项目为准；
- 不在文档中使用无法验证的数据。

11.3 技术依赖风险

如果成员过度依赖AI，可能导致自己并不理解代码和文档内容，在后续答辩、调试和维护时出现问题。

应对策略：

- 成员必须能够解释自己负责模块的设计和代码；
- AI生成代码必须经过本地运行验证；
- 不允许直接提交无法理解的AI生成内容；
- 将AI作为辅助工具，而不是最终责任主体。

11.4 输出质量不稳定风险

AI输出结果受提示词、上下文和工具能力影响，可能出现重复、错误、格式不统一等问题。

应对策略：

- 为AI输出物设置质量标准；
- 对AI生成结果进行人工筛选和修改；
- 对重要内容进行多轮检查；
- 保留人工最终确认节点。

11.5 过程复杂度增加风险

引入AI后，如果流程设计过重，可能反而增加课程项目负担。

应对策略：

- 只在关键环节引入AI；
- 不建立复杂审批流程；
- AI节点服务于效率提升，而不是增加形式化工作；
- 保持课程项目轻量化特点。

十二、改进方案总结

本方案在原有软件过程基础上，引入AI辅助需求分析、概要设计、编码实现、测试验证和文档管理，形成了一套适合“面向网课的智能监督系统”的AI增强型软件过程。

改进后的过程具有以下特点：

1. 保留原有课程项目的软件生命周期主线；
2. 在关键节点加入AI辅助活动；
3. 明确AI输出物质量标准；
4. 新增AI协同负责人等角色职责；
5. 为每个改进环节推荐具体工具；
6. 设置过程度量指标；
7. 对数据安全、AI幻觉、技术依赖等风险提出应对策略。

整体来看，该方案具有较强的可落地性。它不要求团队搭建复杂平台，而是利用现有AI工具提高文档编写、接口设计、代码实现、测试设计和问题分析效率。尤其是AI辅助测试用例生成，与本项目的视频采集与预处理模块高度相关，适合作为后续《AI技术应用验证报告》的核心验证场景。